RESEARCH

Evidence-Guided Crash Adjudication for LLM-assisted Fuzzing of Third-party Open-source Libraries in Connected Vehicles

Qingguan Gao[1*], Ying Zhang[1], Letian Sha[2], Huiran Yang[3]

[1*]School of Cyber Science and Engineering, Southeast University, Nanjing, China.
[2]School of Computer Science, Nanjing University of Posts and Telecommunications, Nanjing, China.
[3]The First Research Institute of the Ministry of Public Security, China.

*Corresponding author(s). E-mail(s): gaoqingguan@seu.edu.cn;
Contributing authors: zhangying@seu.edu.cn; ltsha@njupt.edu.cn;
yanghuiran@zdcst.cn;

Abstract

Connected-vehicle software relies on third-party open-source libraries, making it a relevant application setting for studying crash adjudication in LLM-assisted fuzzing. Advances in Large Language Models (LLMs) have increasingly automated fuzz-driver generation. Nevertheless, directly applying LLMs to raw crash reports can produce many false alerts, while case-by-case triage incurs prohibitive labor costs. To tackle this problem, we propose CrashSage, an evidence-guided framework for post-fuzzing crash adjudication in LLM-assisted fuzzing. Before final LLM analysis, CrashSage prepares crash evidence through three key designs: rule-guided crash attribution, representative selection, and crash-context reconstruction. Rule-guided crash attribution filters low-value reports and likely driver-induced crashes to improve false-alert judgment. Representative selection compares exact and stack signatures among retained reports to identify repeated crash instances and submits only selected representatives to the LLM, reducing redundant analysis. Crash-context reconstruction enriches representatives with static and runtime evidence, helping determine whether the faulty state comes from the generated driver or the target component. Experiments on 1,217 crash reports from 14 third-party open-source libraries relevant to connected-vehicle software show that CrashSage achieves 76.54% precision, 92.56% recall, and an 83.79% F1 score, while reducing LLM submissions from 1,217 to 96 (92.11%). These results support CrashSage as a crash-adjudication aid in this application setting.

Keywords: fuzz testing, crash adjudication, large language model, crash attribution, connected vehicle security

1 Introduction

Intelligent connected vehicle (ICV) systems increasingly depend on third-party open-source libraries (Petit and Shladover 2015; Sun et al. 2022; Bernardini et al. 2017; Checkoway et al. 2011; Ladisa et al. 2023), causing ICV software to inherit security risks from these libraries. To expose crash-inducing defects in such libraries, fuzz testing is widely adopted (Böhme et al. 2019; Lemieux and Sen 2018; Ding and Le Goues 2021). Yet validating diverse third-party open-source libraries at scale still requires a large number of library-specific fuzz drivers. Recent LLM-assisted fuzzing systems attempt to ease this bottleneck by using LLMs to synthesize fuzz drivers automatically (Meng et al. 2024; Deng et al. 2024, 2023; Xia et al. 2024; Xu et al. 2025).

LLM-assisted fuzzing differs from conventional component fuzzing, where the harness is usually written or reviewed by developers and is therefore treated as trusted during crash inspection. In LLM-assisted fuzzing, the generated driver is not necessarily a trusted harness. It invokes target-component APIs with mutated inputs, but it may omit required initialization, construct invalid arguments, or introduce harness-level memory errors. As a result, a crash report no longer reflects target-component behavior alone, but the interaction between the generated driver and the target component.

This difference leads to three challenges in false-alert judgment, crash localization, and efficiency. First, existing crash adjudication tends to treat crash reports as direct evidence of target-component-attributed crashes, leaving driver-induced false alerts difficult to distinguish from target-component-originated crashes. Second, existing adjudication lacks localization evidence that combines static code context and runtime context, which can lead to inaccurate crash attribution. Third, repeated crash instances triggered during fuzzing may stem from the same underlying failure pattern, but existing workflows still analyze them separately, resulting in redundant triage, localization effort, and LLM submissions.

To address this problem, we propose CrashSage, an evidence-guided crash-adjudication pipeline tailored for LLM-assisted fuzzing with incompetent generated drivers. Rather than reasoning directly over raw crash reports, CrashSage applies rule-guided crash attribution to filter low-value reports and likely driver-induced failures, uses representative selection to reduce repeated crash instances, and performs crash-context reconstruction to localize the origin and propagation of the faulty state for the selected representatives. The resulting evidence is then submitted to LLM analysis for final crash diagnosis. In this way, the LLM reasons over filtered, representative, and localized crash evidence. CrashSage is designed as a post-fuzzing triage aid that reduces the number of reports requiring detailed inspection and prepares attribution evidence for retained representative crashes. The main contributions of this paper are listed as follows.

- CrashSage pipeline. We propose an evidence-guided crash-adjudication pipeline for LLM-assisted fuzzing. By preparing structured crash evidence before model reasoning, CrashSage reframes the LLM from a primary classifier over raw crash text into a final analyzer over curated execution evidence.
- Rule-guided crash attribution. We introduce rule-guided crash attribution over sanitizer signals and stack-frame roles. This stage filters low-value reports and likely driver-induced crashes before LLM analysis, mitigating over-attribution bias in prompt-based methods and improving false-alert judgment.
- Representative selection. We compare exact and stack signatures to identify repeated crash instances and retain representative crashes for subsequent LLM analysis. This selection process

reduces redundant analysis while preserving the relationship between each repeated crash and its representative.

- Crash-context reconstruction. We reconstruct static code context and runtime context for representative crashes to trace a crash across driver and component code. By combining source locations with execution-state evidence, this stage supports crash attribution by distinguishing the visible crash site from the origin of the faulty state.
- Experimental evaluation. We evaluate CrashSage using 1,217 crash reports collected from 14 third-party open-source libraries. Compared with the baseline, the full pipeline achieves 76.54% precision, 92.56% recall, and an 83.79% F1 score, while reducing end-to-end LLM submissions from 1,217 to 96 (92.11%). Runtime context reconstruction and LLM analysis are the main sources of latency, while rule-guided crash attribution and representative selection each require less than 10 ms per report.

The rest of the paper is organized as follows. Section 2 reviews related work. Section 3 introduces the LLM-assisted fuzzing framework and its crash-adjudication challenges. Section 4 presents the design of CrashSage, Section 5 reports the evaluation, and Section 7 concludes the paper.

2 Related Work

2.1 LLM-Assisted Fuzzing

Fuzz testing remains a standard way to expose memory-safety defects in C/C++ software, especially with sanitizer support and continuous fuzzing infrastructure (Böhme et al. 2019; Lemieux and Sen 2018; Serebryany et al. 2012; Stepanov and Serebryany 2015; Ding and Le Goues 2021). In conventional component fuzzing, the driver is usually written or reviewed by developers, so downstream analysis can treat it as part of the trusted harness. However, writing such drivers for diverse libraries and APIs requires substantial manual effort, which limits the scalability of component-level fuzzing.

Recent LLM-assisted fuzzing systems address this scalability problem by using models to infer protocol knowledge, generate inputs, synthesize fuzz drivers, and repair compilation errors. ChatAFL (Meng et al. 2024) guides stateful protocol fuzzing with LLM-derived knowledge; FuzzGPT (Deng et al. 2024) and TitanFuzz (Deng et al. 2023) use LLMs for edge-case or test-input generation; Fuzz4All (Xia et al. 2024) generalizes LLM-based input generation across languages; PromptFuzz (Lyu et al. 2024) studies prompt strategies for C/C++ fuzz-driver generation; and CKGFuzzer (Xu et al. 2025) combines code-knowledge-graph construction with LLM-based driver generation and repair. Surveys of LLM-assisted testing likewise emphasize test generation, input synthesis, and repair (Wang et al. 2024). These systems lower driver-construction cost, but their main optimization target remains the upstream fuzzing workflow. Once a generated driver compiles and produces crashes, crash reports are usually treated as fuzzing outcomes rather than as evidence that must be adjudicated under the assumption that the harness may be incompetent. As a result, existing LLM-assisted fuzzing work does not directly address whether a crash report reflects a driver-induced false alert or a target-component-originated crash.

2.2 Automated Crash Diagnosis and Reasoning

Given the post-fuzzing analysis gap left by LLM-assisted fuzzing, a natural question is whether existing crash diagnosis techniques can resolve the resulting crash reports. Early automated diagnosis methods relied on stack-trace signatures and heuristic rules for clustering, localization, and duplicate

detection. Igor (Jiang et al. 2021) clusters fuzz crashes by root cause; FuzzerAid (Joshy and Le 2022) groups crashes using learned fault signatures; CrashLocator (Wu et al. 2014) localizes crashing faults from crash stacks; FaST (Rodrigues et al. 2022) aligns stack traces efficiently; and ReBucket (Dang et al. 2012) clusters duplicate crash reports using call-stack similarity. Related methods enrich crash reports with source, execution, or explanation-oriented evidence: DeepAnalyze (Shetty et al. 2022) localizes crashing faults at scale, CrashTranslator (Huang et al. 2024) reconstructs executable scenarios from stack traces, and Better Debugging (Yan et al. 2024) combines static analysis with LLMs for explainable crash localization.

These techniques provide useful clustering and localization support, but they are most effective when crash evidence is stable and the harness is trusted. This assumption weakens under LLM-generated fuzz drivers. Analyses that do not explicitly localize the failure across driver and component code can mistake a component-visible crash site for a component-originated failure. Localization-oriented methods alleviate part of this ambiguity by recovering static code context, but existing execution enrichment is not designed to bind replay-consistent runtime context to driver/component attribution. Consequently, it may not reveal the runtime values, pointer states, or object states that determine whether the failure originates from driver-side construction or target-component behavior. Likewise, crash clustering can group repeated reports, but it does not by itself decide which representative should receive expensive LLM analysis or how duplicate reports should inherit the representative diagnosis.

LLM-based vulnerability reasoning appears to offer richer semantic judgment for this setting. CVE-LLM enriches vulnerability evaluation with cybersecurity ontologies (Ghosh et al. 2025), while AutoCVSS studies prompting, few-shot examples, and retrieval-augmented generation for CVSS prediction (Sanvito et al. 2025). However, these systems typically reason over curated vulnerability descriptions or structured security context, whereas LLM-assisted fuzzing often yields short crash reports from automatically generated drivers. Direct prompting over such reports lacks the structured semantics assumed by vulnerability-reasoning systems and can amplify over-attribution.

Overall, existing work leaves three gaps for LLM-assisted fuzzing crash adjudication. It does not reliably support false-alert judgment when sanitizer evidence mixes driver behavior with target-component behavior. It lacks attribution-oriented localization that combines static code context and runtime context, making it difficult to determine whether a failure propagates from the generated driver or originates within the target component. It also leaves efficiency concerns when repeated crash instances caused by the same underlying failure pattern are analyzed through separate LLM submissions. This motivates a pipeline that combines rule-guided crash attribution, representative selection, crash-context reconstruction, and LLM-based semantic reasoning before final crash diagnosis.

3 LLM-Assisted Fuzzing Framework and Challenges

3.1 LLM-Assisted Fuzzing Framework

Figure 1 illustrates the overall workflow of the LLM-assisted fuzzing framework considered in this work. The framework automates the process from modeling component APIs to generating fuzz drivers and collecting crash reports, and consists of three stages: API knowledge modeling, driver generation, and crash-oriented fuzzing (Xu et al. 2025).

API knowledge modeling. This stage first extracts the APIs to be tested from open-source components. Static feature extraction then collects key information about the target APIs and identifies

their relationships. The extracted information is organized into an API knowledge graph, which provides structured API knowledge for subsequent combination planning and driver generation (Xu et al. 2025).

Driver generation. The combination agent summarizes the target APIs and uses the API knowledge graph to plan API combinations. Each planned combination, together with the corresponding API source code and summaries, is provided to the driver generation agent, which synthesizes a fuzz driver that invokes the selected APIs (Meng et al. 2024; Xu et al. 2025; Lyu et al. 2024).

Crash-oriented fuzzing. For each generated driver, the seed-generation agent produces initial seeds. The fuzzing engine iteratively mutates these seeds and executes the driver in an attempt to trigger crashes. When a crash occurs, the sanitizer records the failure and exports the corresponding crash report for subsequent crash adjudication (Böhme et al. 2019; Lemieux and Sen 2018; Serebryany et al. 2012; Stepanov and Serebryany 2015).

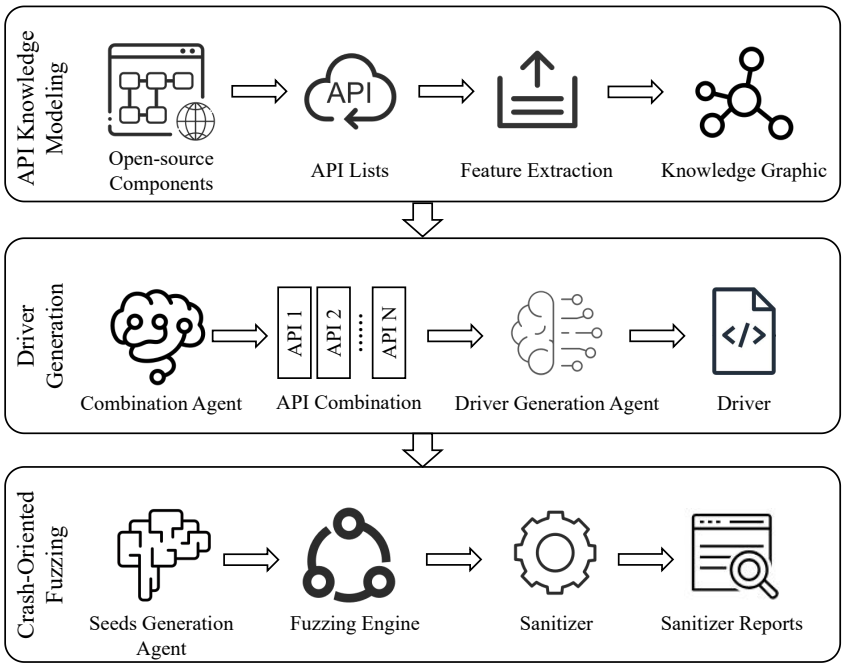


Fig. 1 Overview of the LLM-assisted fuzzing framework.

In the workflow described above, generated drivers are assumed to be incompetent rather than malicious. An incompetent driver may omit required initialization, construct invalid arguments, or otherwise violate API contracts because of unintentional generation errors or insufficient API understanding. We assume that the sanitizer reports, crash artifacts, and source code used for analysis have not been deliberately tampered with. Under these assumptions, rule-guided crash attribution uses sanitizer signals and stack-frame roles to screen likely driver-induced failures. Deliberately malicious drivers, including those that forge evidence or attempt to evade adjudication, are outside the scope of this work.

3.2 Challenges Statement

Unlike conventional fuzzing, LLM-assisted fuzzing cannot assume that the generated driver is a trusted harness (Xu et al. 2025; Lyu et al. 2024). A reported crash may result from the target component, the generated driver, or their interaction under a mutated input. Consequently, the crash report alone may not provide sufficient evidence for reliable post-fuzzing analysis. This setting raises three challenges: false-alert judgment, crash localization, and efficiency.

False-alert judgment. An LLM-generated driver may violate implicit API contracts by omitting required initialization, constructing inconsistent object states, or passing invalid argument combinations. Such misuse can trigger a failure while execution is inside the target component, causing the sanitizer stack trace to contain component frames even when the underlying fault originates from the driver. The presence of a component frame therefore does not by itself establish target-component attribution. Distinguishing driver-induced failures from target-component-originated crashes is difficult because a raw report records where the failure becomes observable, but does not explicitly identify which side violated the required execution contract.

Efficiency. LLM-assisted fuzzing repeatedly mutates related inputs, so many crashing inputs may exercise the same underlying failure pattern. Different generated drivers may also reach that failure pattern through distinct API combinations or call paths. These executions can produce reports that differ in addresses, stack depth, or intermediate frames despite sharing one root cause (Jiang et al. 2021; Joshy and Le 2022; Dang et al. 2012). Treating every report as an independent case leads to repeated inspection and redundant LLM submissions, making crash adjudication increasingly expensive as the number of generated drivers and crashing inputs grows.

Crash localization. Existing prompt-based analysis commonly reasons over raw crash reports without explicitly localizing the failure across the generated driver and the target component. Because a driver-constructed invalid state may become observable only after entering component code, the visible crash frame can be mistaken for evidence that the open-source supply-chain component caused the failure. Localization information is therefore necessary not only to identify a suspicious statement, but also to determine where the faulty state originates and how it reaches the crash site (Shetty et al. 2022; Huang et al. 2024; Yan et al. 2024; Wu et al. 2014). However, existing localization evidence is dominated by static code context, while runtime context is not systematically incorporated into driver/component attribution. Static code context cannot expose the concrete values, pointer states, or object states involved in the failing execution, leaving the propagation of the faulty state incomplete and crash attribution uncertain.

4 Approach

4.1 CrashSage Design Overview

In this section, we present the system design of CrashSage, an evidence-guided crash-adjudication framework for LLM-assisted fuzzing. CrashSage is designed to improve post-fuzzing crash adjudication when automatically generated fuzz drivers cannot be treated as trusted harnesses. Its primary objective is to prepare structured evidence that supports attribution-oriented localization, remove repeated crash instances, and retain auditable inputs before final LLM reasoning. Through this pipeline, CrashSage supports false-alert judgment, crash localization, and efficient analysis of repeatedly triggered crashes.

Figure 2 illustrates the workflow of CrashSage and aligns the three challenges with the corresponding processing stages. Initially, *rule-guided crash attribution* addresses false-alert judgment by distinguishing noise and likely driver-induced crashes from cases that require further target-component analysis. It normalizes crash reports and applies rule-based preliminary screening over sanitizer signals and stack-frame roles, producing filtered normalized crash reports for the following stages. Next, *representative selection* addresses efficiency by avoiding repeated analysis of crashes that follow the same underlying failure pattern. It constructs exact and stack signatures from the normalized sanitizer evidence, verifies signature matches, maps repeated crashes to an existing representative, and retains unmatched crashes in the selected crash set. CrashSage then performs *crash-context reconstruction* to enrich these selected representatives with static code context and replay-consistent runtime context, helping determine where an invalid state originates and how it propagates to the visible crash site. Finally, *LLM analysis* completes the semantic diagnosis by combining each representative crash's normalized crash report, triage label, and reconstructed context to determine its type, cause, location, triggering condition, and severity.

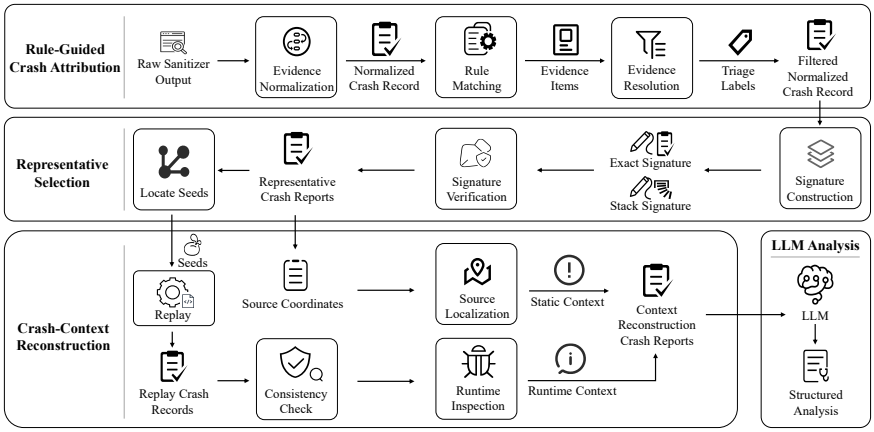


Fig. 2 Multi-stage crash-adjudication pipeline combining rule-guided crash attribution, representative selection, crash-context reconstruction, and LLM analysis.

4.2 Rule-Guided Crash Attribution

Rule-guided crash attribution is the first stage of CrashSage and addresses the false-alert judgment challenge. It consists of sanitizer evidence normalization and rule-based preliminary screening. The former converts raw sanitizer output into a normalized crash report, while the latter assigns a triage label that determines whether the crash is filtered.

4.2.1 Sanitizer Evidence Normalization

Crash reports contain substantial redundant information that is not directly useful for subsequent crash adjudication. CrashSage normalizes each raw crash report into a normalized crash report that retains the information required by subsequent stages. Table 1 summarizes the normalized field groups.

- Failure identity describes the crash report type and its failure category.
- Crash-site metadata records the immediate low-level crash information, such as the failing address or access behavior, when it is available.

Table 1 Key field groups in the normalized crash report

Field Group	Representative Extracted Fields
Failure identity	`sanitizer_type`, `bug_type`, `summary`
Crash-site metadata	`crash_address`, `pc`, `access_type`
Severity and dedup cues	`scariness_score`, `scariness_desc`, `dedup_token`
Structured stack trace	ordered `frames` with function, file, line, and address
Project anchor	`first_in_project_frame`, `first_in_project_function`
Reproduction and status hints	`artifact_path_hint`, `base_unit_hash`, special-condition flags

- Severity and dedup cues provide sanitizer- or fuzzer-generated hints that help prioritize and compare crash reports.
- Structured stack trace converts the raw call stack into ordered frames for later role modeling.
- Project anchor identifies the first visible connection between the crash trace and the target-component source code.
- Reproduction and status hints preserve crash-input clues and special execution states that may affect filtering or replay.

4.2.2 Rule-Based Preliminary Screening

Screening begins by matching the function name, file path, and position of each frame in the `structured stack trace` against predefined project, driver, and runtime patterns. Frames without sufficient matching evidence remain *unclassified*.

Table 2 Stack-frame role classification used by CrashSage

Role	Meaning
project	Stack frames in the target-component source code.
driver	Stack frames related to the fuzz driver.
runtime	Runtime-library frames, including libFuzzer, sanitizers, LLVM runtime, libc, etc.
unclassified	Frames that cannot be classified reliably but are not obvious runtime frames.

Table 2 summarizes these roles. Frame ownership alone is not sufficient for false-alert judgment. A crash trace can involve target-component frames, generated-driver frames, and runtime frames at the same time, while the frame where the sanitizer reports the failure may not be the frame that best explains the origin of the crash. CrashSage therefore applies a set of deterministic rules over the ownership and position of stack frames to infer whether the crash is more consistent with noise, a driver-induced failure, or a target-component crash.

Before listing the rules, we define the following notations. Let $S = \langle f_1, \ldots, f_n \rangle$ denote the structured stack trace, ordered from the top crashing frame to lower caller frames. Let F_P, F_D, and F_N denote the sets of project frames, driver frames, and non-runtime frames, respectively, where $F_N = F_P \cup F_D \cup F_U$ and F_U is the set of unclassified frames. CrashSage computes the driver-frame

ratio

$$R_D = \frac{|F_D|}{\max(|F_N|, 1)},$$

corresponding to `Driver_Frame_Ratio`. Runtime frames are excluded from this ratio because they mainly reflect sanitizer and libFuzzer control flow. Let h_D denote the top-stack depth inspected for driver evidence, θ_D the driver-frame-ratio threshold, m_P the maximum number of project frames treated as shallow project involvement, and θ_S the sanitizer-severity threshold.

CrashSage sets these parameters through an evidence-boundary design strategy that maps each value to an interpretable stack or sanitizer signal. The parameter h_D defines the near-crash stack region used for driver-evidence inspection, θ_D defines the driver-dominance level in the attribution-relevant non-runtime stack, m_P defines the amount of project-code involvement treated as shallow API-boundary execution, and θ_S defines the sanitizer-severity level used to retain project-frame SEGV/null-dereference reports.

For h_D, CrashSage sets $h_D = 3$ because a three-frame window corresponds to the crash frame, the direct caller, and the caller that usually reflects the immediate driver-side invocation context. This value keeps the check centered on the local failure path where argument construction, API invocation, and first observable faulty use are most directly represented. For θ_D, CrashSage sets $\theta_D = 0.8$, using a four-fifths majority as the driver-dominance criterion over non-runtime frames. This value requires driver evidence to occupy a clearly dominant share of the attribution-relevant stack before assigning `likely_driver_bug`, making the label depend on stack-level concentration rather than an isolated driver frame. For m_P, CrashSage sets $m_P = 2$ because shallow API-boundary execution often includes the target entry function and one adjacent helper, initialization, creation, setup, or parsing frame. This value captures brief project-code contact during driver-constructed calls while distinguishing it from deeper target-component execution paths. For θ_S, CrashSage sets $\theta_S = 10$ as the high-severity sanitizer threshold for SEGV/null-dereference reports. This value retains reports when the sanitizer signal indicates severe invalid memory access and the stack simultaneously provides project-frame evidence for target-component analysis.

These values are fixed before evaluating the crash reports, providing a stable and reproducible configuration for the rule-guided attribution stage. The remaining rule signals also come from the normalized crash report. Timeout, `ALARM`, out-of-memory, LSan-only, and deadly-signal flags are extracted from sanitizer or libFuzzer status lines, while `SCARINESS` comes from the sanitizer severity line. Table 3 summarizes the 13 deterministic rules, their predicates, and their preliminary output labels. These labels express the direction supported by the available evidence rather than a final vulnerability verdict.

- R1. A `deadly_signal` marker confirms abnormal termination, but the absence of non-runtime frames leaves no project or driver location for attribution. The report is therefore labeled `noise` because it provides no inspectable evidence for the subsequent analysis.
- R2. Timeout and `ALARM` markers indicate that execution exceeded its time limit, without providing sanitizer evidence for a memory-access crash. Such reports are labeled `noise` because they do not provide crash evidence for component attribution.
- R3. An out-of-memory marker records resource exhaustion, which may depend on allocation demand or execution limits and does not identify a specific faulty access. The rule therefore assigns `noise` rather than attributing the condition to either the driver or the target component.

Table 3 Rules used for crash attribution

Group	Rule	Deterministic Predicate	Output Label
NF	R1	$F_N = \emptyset \wedge$ `deadly_signal`	`noise`
NF	R2	Timeout or `ALARM` marker appears	`noise`
NF	R3	Out-of-memory marker appears	`noise`
NF	R4	LSan-only report without crashing execution	`noise`
DQ	R5	`use_of_uninitialized_value` with a driver frame in the top-h_D stack frames	`likely_driver_bug`
DQ	R6	$R_D \geq \theta_D$	`likely_driver_bug`
DQ	R7	Project init/option/parse function appears near the driver boundary	`likely_driver_bug`
DQ	R8	`stack_buffer_overflow` with the first non-runtime frame in driver code	`likely_driver_bug`
DQ	R9	$F_N = F_D \wedge F_P = \emptyset$	`likely_driver_bug`
DQ	R10	$\lvert F_P \rvert \leq m_P$ and project function matches init/create patterns	`likely_driver_bug`
TC	R11	Memory-safety crash with at least one project frame	`target_crash`
TC	R12	SEGV/null-dereference with `SCARINESS` $\geq \theta_S$ and project frame evidence	`target_crash`
LE	R13	Assertion, abort, or `ABRT` originates in project code	`needs_review`

Note. NF: noise filtering; DQ: driver-quality detection; TC: target-component attribution indication; LE: logic-error indication. F_N, F_D, and F_P denote non-runtime, driver, and project frame sets, respectively.

- R4. An LSan-only report identifies unreleased allocations at process exit but contains no crashing instruction, signal, or abort. Because the rule-guided stage concerns the origin of crashes, this report is labeled `noise`.
- R5. An uninitialized-value report indicates that data was consumed before receiving a valid value. When driver frames occur within the top h_D frames, the first observable use is close to driver-side execution, so the available evidence supports `likely_driver_bug`.
- R6. After runtime frames are removed, $R_D \geq \theta_D$ means that driver frames account for the specified dominant share of the attribution-relevant stack. Because the observable crash path is concentrated in the generated harness, the rule assigns `likely_driver_bug`.
- R7. Initialization, option-processing, and parsing functions near the driver boundary are commonly reached while the generated driver constructs objects and prepares API arguments. When project involvement remains at this boundary, the evidence favors invalid driver-side setup over target-component-originated execution, leading to `likely_driver_bug`.
- R8. For a stack-buffer overflow, the first non-runtime frame identifies the function whose active stack object exceeds its boundary. If this frame belongs to the driver, the overflowing object is localized to driver-side execution, so the rule assigns `likely_driver_bug`.
- R9. If every non-runtime frame belongs to the driver and no project frame is present, the trace contains no observable evidence that execution reached target-component code. The crash is therefore labeled `likely_driver_bug`.
- R10. At most m_P project frames limited to initialization or creation functions indicate shallow component involvement during API setup rather than an internal target execution path. This

pattern is treated as `likely_driver_bug` because the evidence primarily reflects how the generated driver configured the call.

- R11. A sanitizer-reported memory-safety crash with at least one project frame establishes that the erroneous execution reached target-component code. The rule labels it `target_crash` so that it is retained as a target-component candidate for subsequent stages, rather than making the final attribution decision in the rule-guided stage.
- R12. A severe SEGV or null dereference indicates an invalid memory access, while the presence of project frames anchors that access in target-component execution. Their combination supports the preliminary label `target_crash` and retains the crash for localization.
- R13. A project-origin assertion, abort, or `ABRT` identifies the component location that terminates execution, but the report alone cannot distinguish a driver violation of an API precondition from a violated internal invariant. The rule therefore assigns `needs_review` rather than attributing the crash to either side.

Because the rules are evaluated independently, CrashSage resolves their evidence items in the order `noise`, `likely_driver_bug`, `target_crash`, and `needs_review`. The first matched label in this order becomes the triage label; if no rule matches, the crash is assigned `needs_review` by default. This procedure removes reports without usable crash evidence and generated-driver crashes before retaining target-component candidates and ambiguous cases for further analysis.

Crashes labeled `noise` or `likely_driver_bug` are filtered, whereas those labeled `target_crash` or `needs_review` are kept as retained normalized crash reports for representative selection. For a crash c, let $\mathcal{N}(c)$ denote its normalized crash report and $y(c)$ denote its triage label. The output of rule-guided crash attribution is the pair $\langle \mathcal{N}(c), y(c) \rangle$, which serves as the input to the subsequent stage.

4.3 Representative Selection

Large-scale fuzzing often produces many crash instances that differ in inputs or generated drivers but follow the same underlying failure pattern. Representative selection reduces this repeated analysis cost by comparing exact and stack signatures and forwarding only non-duplicate representatives to the later LLM stage.

Let $\mathcal{C} = \{c_1, \ldots, c_n\}$ denote the set of crashes retained after rule-guided crash attribution. For each crash c, let S_c denote its structured stack trace following the definition of S in Section 4.2, and let $d(c)$ denote the sanitizer deduplication token. Let $b(c)$ denote the sanitizer bug type extracted from $\mathcal{N}(c)$, and let $\ell(c)$ denote the crash-site source location derived from the crash-site metadata or resolved frame location when available. Let $\perp$ denote an unavailable value. The sequence $s(c)$ is the ordered non-runtime key-frame subsequence derived from S_c. Runtime frames are excluded from $s(c)$ because they usually describe sanitizer, libFuzzer, or libc control flow rather than the project-driver execution path.

Exact signature. The exact signature targets crashes with the same sanitizer-provided identity or the same resolved crash site. CrashSage computes

$$\sigma_{\text{exact}}(c) = \begin{cases} H(d(c)), & d(c) \neq \perp, \\ H\big(b(c), \ell(c), \text{top}(s(c))\big), & d(c) = \perp \wedge \ell(c) \neq \perp \wedge |s(c)| > 0, \\ \perp, & \text{otherwise}, \end{cases}$$

where $H(\cdot)$ is a stable hash function and $\mathrm{top}(s(c))$ denotes the first frame in the non-runtime key-frame sequence. The exact-signature match between two crashes is defined as

$$M_{\mathrm{exact}}(c_i, c_j) = \mathbb{I}\big[\sigma_{\mathrm{exact}}(c_i) \neq \bot \wedge \sigma_{\mathrm{exact}}(c_i) = \sigma_{\mathrm{exact}}(c_j)\big].$$

Here, $M_{\mathrm{exact}}(c_i, c_j) = 1$ represents the strongest duplicate match. The condition holds either because both crashes expose the same sanitizer deduplication token $d(c)$, or because the normalized fields $b(c)$, $\ell(c)$, and $\mathrm{top}(s(c))$ agree. In the latter case, the two reports have the same sanitizer crash class, the same localized crash-site anchor such as file, line, and function, and the same first project/driver frame after runtime frames are removed.

Algorithm 1 Representative Selection

Require: retained crashes $\mathcal{C} = \langle c_1, \ldots, c_n \rangle$ in discovery order
Ensure: selected crash set $\mathcal{R}$ and duplicate mapping $\mathcal{L}$

```
R ← ∅, L ← ∅
for c_i ∈ C do
    r* ← ⊥
    for r ∈ R do
        if M(c_i, r) = 1 then
            r* ← r
            break
        end if
    end for
    if r* ≠ ⊥ then
        L ← L ∪ {(c_i, r*)}
    else
        R ← R ∪ {c_i}
    end if
end for
return (R, L)
```

Stack signature. The stack signature targets crashes whose crash-site anchors differ but whose project-driver call paths are identical. Let $p_k(c)$ denote the ordered sequence obtained by taking the first $\min(k, |s(c)|)$ frames from $s(c)$, preserving the original stack order from the top crashing frame to lower caller frames. Here, k is a fixed prefix length used consistently for all crashes. CrashSage builds a stack signature from this bounded prefix:

$$\sigma_{\mathrm{stack}}(c) = H\big(b(c), p_k(c)\big).$$

The bug type is kept in the signature to avoid treating different sanitizer crash classes that happen to pass through similar functions as duplicates. The bounded prefix of the non-runtime stack captures the driver-to-project execution path that reaches the crash. The stack-signature match is defined as

$$M_{\mathrm{stack}}(c_i, c_j) = \mathbb{I}\big[\sigma_{\mathrm{stack}}(c_i) = \sigma_{\mathrm{stack}}(c_j)\big].$$

Here, $M_{\text{stack}}(c_i, c_j) = 1$ represents a duplicate match based on the retained call path rather than on an identical crash-site anchor. The equality of σ_{stack} requires the same $b(c)$ and the same bounded prefix $p_k(c)$, where $p_k(c)$ is built from the ordered non-runtime frames in $s(c)$. Thus, the two crashes are treated as duplicates when their normalized stack evidence shows the same sanitizer crash class and the same leading project/driver frame sequence, even if the reported line number, resolved crash location, or exact sanitizer token is unavailable or slightly different.

CrashSage evaluates the two match predicates in the order of exact and stack matching, which corresponds to the signature-verification step of the pipeline overview in Section 4.1. For a pair of crashes, stack matching is evaluated only when exact matching does not produce a match. The resulting duplicate decision is

$$M(c_i, c_j) = M_{\text{exact}}(c_i, c_j) \vee M_{\text{stack}}(c_i, c_j).$$

CrashSage processes the retained crashes in discovery order and compares each crash with the representatives selected so far. A matched crash is linked to the corresponding representative, whereas an unmatched crash becomes a new representative. Algorithm 1 summarizes this procedure.

The procedure therefore does not construct equivalence classes through a transitive closure. A crash is associated with the first matching representative, and two crashes that match the same representative are not thereby declared equivalent. Each crash is mapped to at most one representative, and the mapping is stored explicitly rather than inferred from the match relation.

The primary output of representative selection is the selected crash set $\mathcal{R}$. The accompanying mapping $\mathcal{L}$ records each duplicate–representative pair (c_i, r), indicating that c_i matches and reuses the analysis result of representative r.

4.4 Crash-Context Reconstruction

After representative selection, the selected crash representatives still require relevant localization evidence to refine the attribution. A component frame identifies where a failure becomes visible, but it does not establish whether the faulty state originates in the generated driver or the target component. Crash-context reconstruction addresses this gap through static code context reconstruction and runtime context reconstruction. Static code context reconstruction maps the driver-to-component call path to source code, while runtime context reconstruction recovers the execution state needed to explain how the failure propagates along that path. Together, they turn an isolated sanitizer crash label into attribution-oriented execution evidence for LLM adjudication of the selected representatives.

4.4.1 Static Code Context Reconstruction

Static code context reconstruction localizes selected project and driver frames in a representative crash report to source files and extracts the surrounding source-code context. CrashSage first maps each selected frame to a local file, function, and line number using the reported source coordinates. Based on these locations, it extracts bounded code windows around the crash-site frame and relevant call-chain frames, preserving the local statements before and after each key frame. These code windows provide source-level evidence for inspecting the crashing operation and tracing how the fuzz input moves from the generated driver into the target API and toward the crash site. However, static code context alone cannot determine the concrete values and object states present in the failing execution.

4.4.2 Runtime Context Reconstruction

Runtime context reconstruction complements static code context reconstruction by recovering execution-state evidence that static code alone cannot provide, including variable values, pointer states, and memory-related conditions at the crash point. This evidence helps determine whether an invalid state is constructed by the driver and propagated into component code or produced within the component itself. CrashSage uses `artifact_path_hint` or `base_unit_hash` in the normalized crash report to locate the triggering seed and stage an isolated copy for replay.

Replaying the seed is necessary to observe the execution state at the crash point. However, environmental differences, timing-dependent behavior, uninitialized state, or external resources may cause the same seed to produce a different crash. CrashSage therefore parses the replayed crash report into a replay-produced normalized crash record and verifies its consistency before collecting runtime context. To bind the replay result to the representative crash, CrashSage builds a replay key

$$\kappa(c) = \langle b(c), \ell(c), s(c) \rangle,$$

where $b(c)$ corresponds to the `bug_type` field of crash report c, $\ell(c)$ denotes the crash-site source location derived from the crash-site metadata or resolved frame location when available, and $s(c)$ denotes the ordered non-runtime key-frame sequence derived from the structured stack trace S. Let c_0 denote the original normalized crash report and c_r denote the replay-produced crash record. CrashSage regards their replay keys as compatible, denoted by $\kappa(c_r) \simeq \kappa(c_0)$, when

$$b(c_r) = b(c_0) \wedge s(c_r) = s(c_0) \wedge \big(\ell(c_0) = \bot \vee \ell(c_r) = \ell(c_0)\big),$$

where $\bot$ denotes an unavailable value in the corresponding field. Only a replay satisfying this condition is treated as consistent with the representative crash and used for runtime inspection.

For a replay-consistent crash, CrashSage invokes GDB as the runtime inspection tool. The bounded GDB run records the reproduced backtrace and crash frame, together with selected registers, function arguments, and local variables from the frames nearest to the crash site. These fields can help reveal state that static code context cannot provide, such as whether a pointer is null, whether a size variable is inconsistent with a buffer, or whether an object has an invalid state before the crashing operation. Algorithm 2 summarizes this reproduction-gated runtime context reconstruction procedure. The detailed organization of the evidence produced by crash-context reconstruction is presented in Table 4.

LocateSeed, StageSeed, ReplayAndParse, and CollectDebugContext correspond to seed recovery, isolated replay preparation, replay-output parsing, and GDB-based bounded runtime-state collection, respectively. For each representative crash $c \in \mathcal{R}$, the output $C_{\text{recon}}(c)$ contains static code context, optional runtime context, and a status ρ indicating the outcome of seed recovery, replay matching, and GDB collection. If runtime evidence is unavailable, $C_{\text{runtime}} = \bot$, and the static context remains available to subsequent stages.

For each representative crash $c \in \mathcal{R}$, crash-context reconstruction outputs $C_{\text{recon}}(c)$. After this stage, the evidence package consumed by the final LLM analysis is defined as

$$\mathbf{x}(c) = \langle \mathcal{N}(c),\, y(c),\, C_{\text{recon}}(c) \rangle\,.$$

This package combines the normalized crash report and triage label from rule-guided attribution with the reconstructed context produced in this stage.

Table 4 Evidence groups produced by crash-context reconstruction

Context Type	Evidence Group	Representative Fields	Concrete Role
Static code context	Crash-site anchor	`crash_file`, `crash_line`, `crash_function`, `crash_code_snippet`	Locates the visible crash site, not the fault origin.
	Call-chain source context	`call_chain_context`, `frame_type`, `code_snippet`	Preserves bounded project/driver frame context.
Runtime context	Seed recovery metadata	`seed_locate`, `seed_copy`, `seed_paths.seed`	Records how the artifact is recovered and staged.
	Replay consistency	`repro.runs`, `signature`, `same_signature`, `repro_rate`	Audits replay against the representative crash.
	Runtime execution state	`reproduced`, `backtrace_text`, `registers`, `crash_frame`, `call_stack`	Supplies runtime state for tracing propagation.

Algorithm 2 Runtime Context Reconstruction

Require: normalized crash report c_0, static context C_{static}, target binary B, artifact directory D
Ensure: reconstructed context $C_{\text{recon}}(c_0) = (C_{\text{static}}, C_{\text{runtime}}, \rho)$

$u \leftarrow \textsc{LocateSeed}(c_0.\texttt{artifact_path_hint}, c_0.\texttt{base_unit_hash}, D)$
if u is unavailable **then**
  return $(C_{\text{static}}, \perp, \textsc{SeedUnavailable})$
end if
$u_s \leftarrow \textsc{StageSeed}(u)$
$c_r \leftarrow \textsc{ReplayAndParse}(u_s, B)$
if c_r is unavailable **then**
  return $(C_{\text{static}}, \perp, \textsc{ReplayFailed})$
end if
if $\kappa(c_r) \not\simeq \kappa(c_0)$ **then**
  return $(C_{\text{static}}, \perp, \textsc{CrashMismatch})$
end if
$C_{\text{runtime}} \leftarrow \textsc{CollectDebugContext}(u_s, B)$
if C_{runtime} is unavailable **then**
  return $(C_{\text{static}}, \perp, \textsc{RuntimeContextUnavailable})$
end if
$\rho \leftarrow \textsc{RuntimeContextAvailable}$
return $(C_{\text{static}}, C_{\text{runtime}}, \rho)$

4.5 LLM Analysis

Given the evidence package $\mathbf{x}(c)$, CrashSage invokes the LLM only for representative crashes $c \in \mathcal{R}$. The LLM produces a structured diagnosis through schema-guided, evidence-guided prompting rather than direct classification over raw sanitizer text.

Table 5 Prompt design for LLM analysis

Prompt Part	Prompt Design
System role	Act as a crash-adjudication assistant and distinguish the visible crash site from the origin of the faulty state.
Attribution rule	Do not assign target-component attribution unless target-component evidence is stronger than the driver-misuse hypothesis.
Input evidence	Use the triage summary, matched rules, frame-role evidence, normalized sanitizer fields, static context, and runtime context.
Final attribution	For every representative crash, compare the target-component-defect and driver-misuse hypotheses before making the final target-component attribution. Attribute the crash to the target component only when the target-component-defect hypothesis is better supported by the supplied evidence.
Output schema	Return `is_api_bug`, `root_cause_type`, `root_cause_location`, `triggering_condition`, `severity`, `fix_suggestion`, and `evidence_rationale`.
Decoding and retry	Use `temperature=0`; parse the primary response and apply repair only when validation fails.

Table 5 summarizes the prompt design sent to the LLM for each representative crash. For each representative, CrashSage serializes $\mathbf{x}(c)$ into this staged prompt, which we call the differential prompt. Runtime evidence is injected when replay succeeds; otherwise, the prompt records the unavailable runtime status instead of hiding the missing evidence.

Before making the final target-component attribution, the differential prompt asks the model to compare the driver-misuse hypothesis and the target-component-defect hypothesis. The driver-misuse hypothesis considers whether the generated driver caused the observed failure. It is supported when missing required initialization, an inconsistent object state, or an invalid argument combination violates a required API precondition and explains the observed failure. In this case, the visible crash location may be inside the target component. However, the faulty state originates in the generated driver.

The target-component-defect hypothesis considers whether target-component behavior caused the faulty state. It is supported when the generated driver satisfies the relevant API preconditions and the reconstructed evidence indicates that the faulty state originates in target-component behavior. In this case, the target component both contains the visible crash location and produces the faulty state. The model makes the final target-component attribution only when the target-component-defect hypothesis is better supported by the supplied evidence. This differential framing keeps driver misuse in scope and reduces the tendency to over-attribute a component-visible crash to the target component. The Clean direct prompt instead asks for the final attribution directly. CrashSage uses the differential prompt, and both styles are evaluated in Section 5.

The model output is required to follow a structured schema with the fields `is_api_bug`, `root_cause_type`, `root_cause_location`, `triggering_condition`, `severity`, `fix_suggestion`, and

`evidence_rationale`. These fields correspond to the following diagnosis tuple:

$$\mathbf{z}(c) = \langle a(c),\, \tau(c),\, \lambda(c),\, \gamma(c),\, \nu(c),\, \phi(c) \rangle\,,$$

where $a(c)$ denotes the final target-component attribution judgment, $\tau(c)$ denotes the root-cause type, $\lambda(c)$ denotes the root-cause location, $\gamma(c)$ denotes the triggering condition, $\nu(c)$ denotes the severity, and $\phi(c)$ denotes the repair suggestion. The `evidence_rationale` field records the short evidence chain supporting these schema fields and remains part of the report metadata. If validation still fails, the deterministic evidence package is preserved and the LLM diagnosis is marked incomplete. The final structured report is defined as

$$\mathbf{r}(c) = \langle \mathbf{x}(c),\, \mathbf{z}(c) \rangle\,.$$

This report preserves the deterministic evidence prepared by CrashSage together with the LLM-generated semantic diagnosis.

5 Evaluation

To evaluate the performance of CrashSage, we defined four research questions:

- RQ1: False-alert judgment: How effectively does rule-guided crash attribution distinguish driver-induced crashes from target crashes?
- RQ2: Efficiency: How much redundant analysis does representative selection remove, and what latency do the major pipeline operations introduce?
- RQ3: Crash localization: How does crash-context reconstruction improve crash attribution by distinguishing the visible crash location from the origin of the faulty state?
- RQ4: Final analysis sensitivity: When the evidence package and representatives are held fixed, how do the final LLM model and final prompt design affect crash attribution?

5.1 Experimental Setup

CrashSage is implemented in Python and uses `qwen3-coder-flash`, accessed through a cloud API, as the final LLM model. All experiments are conducted on a machine running Ubuntu 22.04 and equipped with an AMD Ryzen 5 5600G with Radeon Graphics processor. We evaluate CrashSage on 14 third-party open-source libraries relevant to connected-vehicle software functions. The selected libraries include c-ares, cJSON, dlt-daemon, expat, faad2, isotp-c, jansson, json-c, libmad, libmspack, libpcap, libsndfile, tinyxml2, and zlib, covering connected-vehicle software functions such as networking, structured-data parsing, packet capture, compression, media decoding, archive processing, in-vehicle transport, and diagnostic logging. The evaluation focuses on the third-party-library layer of connected-vehicle software rather than on a complete vehicle software stack.

Table 6 summarizes the functions, API scales, and representative deployment roles of the evaluated libraries. We collect crash reports following the fuzz-driver generation and fuzzing workflow of CKGFuzzer (Xu et al. 2025). This workflow is adopted because CKGFuzzer couples driver generation with a crash-analysis stage, so the adjudication pipeline can be extended without replacing the upstream fuzzing process; systems that target only input or driver generation do not expose a comparable crash-analysis interface. The generated drivers are compiled and executed by libFuzzer with ASan+UBSan instrumentation in Docker containers based on the OSS-Fuzz base-runner image. The testing process produces 1,217 crash reports in total, all of which are used in the main evaluation.

Table 6 Third-party open-source libraries used in the evaluation.

Library	Function	APIs	Crash Reports	Deployment Role
c-ares	Asynchronous DNS resolution	27	103	Gateway DNS
cJSON	JSON parsing and generation	60	78	Config/telemetry parsing
dlt-daemon	Diagnostic logging	13	92	In-vehicle diagnostic logging and fault-record storage
expat	XML parsing	12	36	Vehicle configuration and metadata parsing
faad2	AAC audio decoding	12	151	In-vehicle AAC and media decoding
isotp-c	ISO-TP multi-frame transport	13	104	ISO-TP transport between ECUs and vehicle modules
jansson	JSON parsing and generation	12	128	ECU configuration and telemetry serialization
json-c	JSON parsing and generation	12	40	Configuration and telemetry processing in vehicle software
libmad	MPEG audio decoding	12	112	In-vehicle MPEG audio decoding
libmspack	Archive extraction	12	54	Vehicle software package and archive processing
libpcap	Packet capture and filtering	30	115	In-vehicle network monitoring and packet capture
libsndfile	Audio file processing	12	76	Audio and log-signal processing
tinyxml2	XML parsing	13	63	Vehicle configuration and metadata parsing
zlib	Compression and decompression	29	65	OTA-update and diagnostic-log compression
Combined	–	269	1,217	–

All 14 libraries are publicly available and can be built under ASan+UBSan within the CKGFuzzer workflow. We characterize exercised API diversity by recording multi-API calls and ordered lifecycle calls involving initialization, creation, opening, allocation, receiving, use, and cleanup. The 14 projects contain 186 generated drivers, between 7 and 22 per project, and the 1,217 reports are distributed over them at 6.50 reports per driver on average. The median number of distinct target APIs per driver ranges from 2 to 8 across projects, 179 of the 186 drivers (96.20%) call at least two distinct target APIs, and 168 drivers (90.30%) also include at least one initialization or cleanup call. Faults in these components may affect diagnostic records, ECU-to-module communication, configuration or telemetry processing, update and log handling, and in-vehicle media services.

Stateful API sequences in isotp-c and dlt-daemon extend the component-level evaluation to transport and logging lifecycles. Vehicle-level timing failures and concurrent resource contention

require separate system-level validation. CrashSage provides crash-attribution evidence for further engineering analysis; ISO 26262 compliance assessment and certification remain distinct from the component-level evaluation reported here.

Table 7 Evaluation configurations.

Variant	Base configuration	Rule-guided filtering	Representative selection	Reconstruction context	Final prompt
V0 (CKGFuzzer baseline)	Yes	No	No	No	Clean direct prompt
V1	Yes	Yes	No	No	Clean direct prompt
V2	Yes	Yes	Yes	No	Clean direct prompt
V3	Yes	Yes	Yes	Yes	Clean direct prompt
V4 (CrashSage)	Yes	Yes	Yes	Yes	Differential prompt

Table 7 summarizes the evaluated variants. Relative to V0, V1 adds rule-guided filtering; V2 further adds representative selection; V3 further adds reconstruction context; and V4 further adopts the complete differential prompt design described in Section 4.5. Thus, V4 combines all proposed components and represents the full CrashSage configuration. Under V3, the clean direct prompt requests the final target-component attribution directly. In contrast, V4 follows the *Final attribution* design: before making the final attribution, the LLM explicitly compares the target-component-defect hypothesis with the driver-misuse hypothesis and assigns the crash to the target component only when the former is better supported by the supplied evidence. Thus, the V3–V4 comparison evaluates the effect of the complete differential prompt design, whereas V4 represents the full CrashSage variant. Per-library tables are reported for V0–V1, V2–V3, and V3–V4.

All variants share the same base configuration, which is the crash-analysis stage of CKGFuzzer: the crash report, the generated driver and the APIs it calls, the error patterns extracted from their source, and the retrieved CWE entries. V0 uses only this configuration. V1–V4 keep it unchanged and add the stages listed in Table 7. The V0–V4 comparison therefore measures the incremental effect of these added stages while holding the fuzz-driver generation and fuzzing workflow of CKGFuzzer fixed and evaluating all variants on the same crash reports.

Table 8 Ground-truth accounting for the final binary evaluation.

Binary label	Reports
Target	215
Non-target	1,002
Total evaluated reports	1,217

Each crash report receives a binary evaluation label: target or non-target. A target report denotes a crash that is locally reproducible, is attributable to target-component API behavior rather than generated-driver misuse, and corresponds to a vulnerability verified through public evidence such as a CVE record, official advisory, or public issue or patch record. A non-target report denotes a crash assigned the negative label under the evaluation protocol because it is not reproducible, lacks

sufficient evidence of a target-component defect after inspection, or is manually confirmed as driver-induced. The final evaluation contains 215 target reports and 1,002 non-target reports across all 1,217 reports, as summarized in Table 8. Labels were assigned by two reviewers with C/C++ memory-safety analysis experience, who examined blinded evidence independently, and disagreements were resolved by a third reviewer. Precision, recall, and F1 are computed over this complete report population.

The 876 reports of the original submission and the 196 reports added in this revision were verified against CVEs and related public vulnerability material. For each report, the correspondence with the public evidence was established by checking the target version, the crash signature, the stack trace path, and the triggering input. A non-target label was determined from the verification of the relevant code and of the triggering process, and not from the mere absence of a matching CVE.

For the 145 reports that lacked sufficient public vulnerability evidence, we conducted an additional manual audit. Three people with C/C++ memory-safety analysis experience took part. Two of them annotated independently without seeing CrashSage outputs: they reproduced the crash from the triggering input, checked the crash signature and the stack trace path, and, using the API contracts and the relevant source code, judged whether the crash exposes a vulnerability in the target library. Their initial annotation agreement was 70.97% with a Cohen's kappa of 0.413. The third person re-examined every report on which the two annotations disagreed, reviewed the basis of both annotations, and performed the necessary code inspection and reproduction to determine the final label. In total, 26 reports were labeled target and 119 were labeled non-target.

The 876 reports of the original submission, the 196 reports added in this revision, and the 145 audited reports therefore account for all 1,217 reports. No report is excluded from the binary metrics, and all experimental configurations use this same fixed ground truth.

5.2 Comparison with Raw-Only LLM Analysis

To evaluate raw-report analysis, we compare a raw-only LLM baseline with the complete CrashSage V4 on the 1,217 crash reports included in the main evaluation. The raw-only baseline passes each original sanitizer report to `qwen3-coder-flash` at temperature 0 with a minimal attribution instruction, without filtering, representative selection, reconstructed context, or the differential prompt.

Table 9 End-to-end comparison between raw-only LLM analysis and complete CrashSage V4.

Method	LLM submissions	Precision	Recall	F1
Raw-only LLM	1,217	0.2010	0.5488	0.2943
CrashSage V4	96	0.7654	0.9256	0.8379

Table 9 reports the end-to-end comparison. Relative to the raw-only baseline, CrashSage V4 improves precision from 0.2010 to 0.7654, recall from 0.5488 to 0.9256, and F1 from 0.2943 to 0.8379, while reducing LLM submissions from 1,217 to 96. Raw-only analysis thus produces many false alerts and misses target-component crashes, while CrashSage reduces both types of attribution error.

5.3 Evaluation of Rule-Guided Crash Attribution

To evaluate rule-guided crash attribution, we compare V1 with V0 on the 1,217 crash reports included in the main evaluation. Reports labeled `target_crash` or `needs_review` are forwarded to subsequent analysis after reports labeled `noise` or `likely_driver_bug` are filtered. Precision measures the proportion of crashes with `is_api_bug=true` that are labeled true, recall measures the proportion of target crashes finally identified by the variant, and F1 reflects their balance.

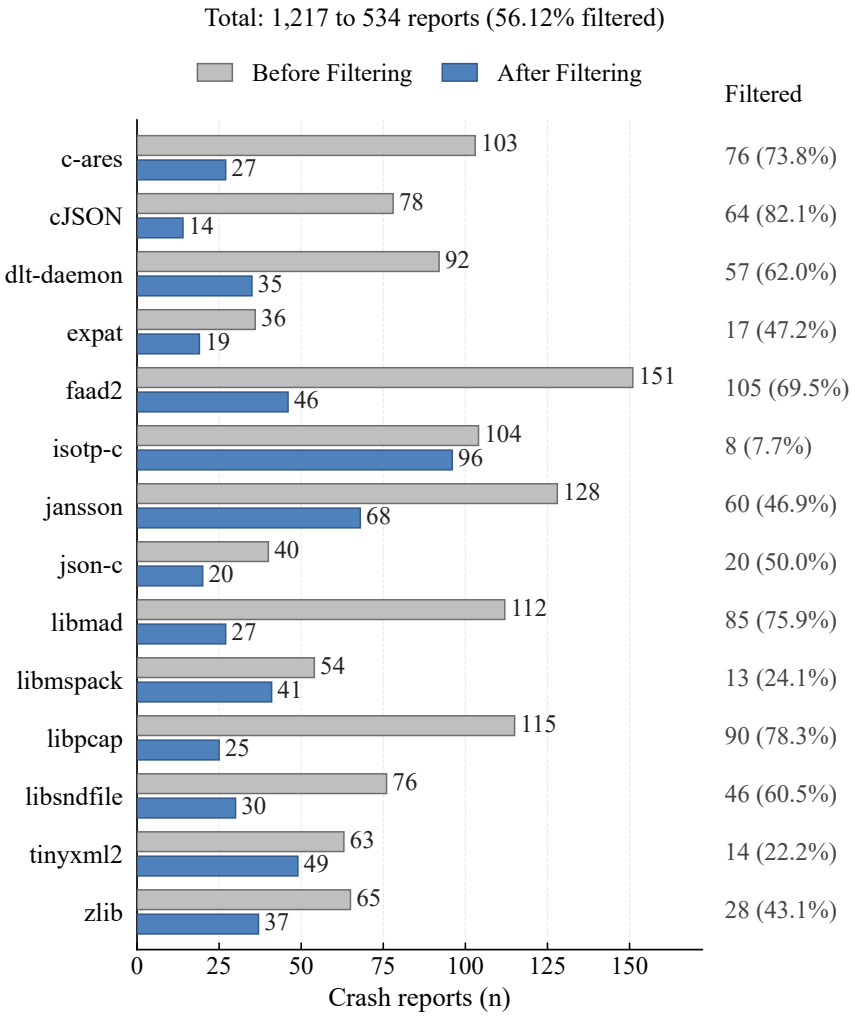


Fig. 3 Reduction in crash reports after rule-guided crash attribution.

The filtered and retained rows together cover all 1,217 reports, of which 215 are target and 1,002 are non-target.

The experimental results are shown in Figure 3 and Tables 10, 11, and 12. Rule-guided crash attribution filters 683 reports from 1,217 crash reports and retains 534 reports for subsequent analysis.

Table 10 Ground-truth audit of rule-guided filtering.

Triage Outcome	Ground-truth True	Ground-truth False	Total
`noise`	0	103	103
`likely_driver_bug`	0	580	580
Filtered	0	683	683
Retained	215	319	534

Table 11 Effect of rule-guided crash attribution.

Variant	TP	FP	TN	FN	Precision	Recall	F1
V0	151	385	617	64	0.2817	0.7023	0.4021
V0|N	168	424	578	47	0.2838	0.7814	0.4164
V1	155	108	894	60	0.5894	0.7209	0.6485

Table 10 shows that all filtered reports are labeled false, whereas all 215 reports labeled true are retained. Thus, rule-guided filtering introduces no ground-truth false negatives at the filtering stage. V1 improves precision and F1 over V0 from 0.2817 and 0.4021 to 0.5894 and 0.6485. Its 60 false negatives are retained reports misclassified by the subsequent LLM rather than reports removed by the filtering rules.

This result is partly attributable to the reports removed by rule-guided crash attribution. Reports with insufficient crash evidence or evidence more consistent with driver-induced failures can introduce misleading information into subsequent analysis. Filtering these reports reduces driver-side cases entering later LLM analysis while preserving target-component candidates that warrant further analysis.

The project-level results indicate that the aggregate improvement is not uniform. The affected projects form two distinct patterns: in cJSON, expat, and zlib, the evidence includes a driver-side assertion or an invalid value supplied by the driver. V0 tends to place greater emphasis on the crash site in target code. In V1, the normalized report and rule-derived evidence make the driver-side condition that determines the validity of the operation more apparent. Consequently, several reports classified as target cases in V0 are classified as driver cases in V1. This reduces target recognition and accounts for the lower F1 in these projects. cJSON decreases from 0.0833 to 0.0000, expat from 0.4762 to 0.2500, and zlib from 0.4516 to 0.2222. isotp-c exhibits the opposite pattern. The invalid bit index originates from the driver, whereas the crash occurs in a target-side bitfield operation. V1 places greater emphasis on the target code containing the failing access. This increases recall to 1.0000, but false positives increase from 15 to 33, resulting in a decrease in F1 from 0.4138 to 0.3265. These paired outcomes show that V1 does not uniformly increase target recognition. The effect of the rule-guided crash attribution depends on which side provides the more decisive evidence for the origin of the invalid state.

To isolate the contribution of crash-report normalization, we further evaluate V0|N, which retains V0's original input and adds only the normalized crash representation. V0|N increases recall from

Table 12 Per-library effect of rule-guided crash attribution.

Library	V0			V1		
	Prec.	Recall	F1	Prec.	Recall	F1
c-ares	0.1290	1.0000	0.2286	0.5000	1.0000	0.6667
cJSON	0.0476	0.3333	0.0833	0.0000	0.0000	0.0000
dlt-daemon	0.7442	0.9143	0.8205	1.0000	1.0000	1.0000
expat	0.6250	0.3846	0.4762	0.6667	0.1538	0.2500
faad2	0.2267	0.5667	0.3238	0.8947	0.5667	0.6939
isotp-c	0.2857	0.7500	0.4138	0.1951	1.0000	0.3265
jansson	0.2105	0.9231	0.3429	0.4000	0.9231	0.5581
json-c	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
libmad	0.1757	1.0000	0.2989	1.0000	1.0000	1.0000
libmspack	0.5667	0.9444	0.7083	1.0000	0.8333	0.9091
libpcap	0.4000	0.7368	0.5185	0.8421	0.8421	0.8421
libsndfile	0.4545	0.3846	0.4167	0.5200	1.0000	0.6842
tinyxml2	0.3939	1.0000	0.5652	0.5000	1.0000	0.6667
zlib	0.3889	0.5385	0.4516	0.2143	0.2308	0.2222

0.7023 to 0.7814 and identifies 17 additional target reports, while precision remains essentially unchanged. In V0, crash type, access behavior, stack order, and project location are embedded in a long and redundant sanitizer report. The model must first distinguish relevant evidence from addresses and repeated runtime messages. V0|N presents these fields in a consistent structure and preserves their order. This reduces ambiguity in the execution path, allowing the model to connect the driver call to the target operation more reliably and recover target cases that V0 misses.

The comparison between V0|N and V1 can be attributed to two distinct stages. Relative to V0|N, V1 achieves higher precision and F1 but lower recall. The precision difference is primarily a filtering effect, because rule-guided crash attribution removes 683 non-target reports before the final judgment and thereby eliminates a substantial number of false-positive opportunities. The recall difference arises at the subsequent judgment stage. Since rule-guided crash attribution removes only non-target reports, the reduction in recall is associated with decisions on the retained target cases. When the driver contains an explicit assertion or provides an invalid value, V1 assigns greater weight to driver responsibility. This explains the reduced recall in cJSON, expat, and zlib. In target-side memory operations reached through invalid driver inputs, V1 assigns greater weight to the target crash site, which increases target detections but also introduces additional false positives, as observed in isotp-c. These effects jointly explain the overall trade-off. The structured report improves target coverage, filtering provides the principal gain in precision, and the rule-derived evidence changes the final assignment of responsibility between the driver and the target.

In response to RQ1, rule-guided crash attribution distinguishes driver-induced crashes from target crashes under the ground-truth labels.

5.4 Evaluation of Representative Selection

To evaluate the efficiency of representative selection, we compare V1 and V2. Both variants apply rule-guided crash attribution to the same crash reports. V1 submits all reports retained after rule-guided crash attribution to the LLM, whereas V2 further applies representative selection before LLM analysis. We first report the runtime of the stages relevant to this comparison and then quantify the reduction in LLM submissions introduced by representative selection.

As shown in Table 13, both stages relevant to the V1–V2 comparison are lightweight. Rule-guided crash attribution uses parsed sanitizer fields and stack-frame roles, whereas representative selection compares crash signatures without replaying the target binary. The measured cost of representative selection is 0.01 ms per retained report, so the reduction in LLM submissions does not introduce a comparable computational overhead before final analysis. Crash-context reconstruction and LLM analysis, by contrast, dominate the mean processing time per representative. Reconstruction includes crash record lookup, seed recovery and binary resolution, static context extraction, direct replay, and Docker/GDB context collection. Crash record lookup locates the archived crash record and loads its metadata, seed recovery locates the crashing input, and binary resolution identifies the corresponding executable. These five substeps take 0.32 s, 0.18 ms, 1.18 ms, 2.44 s, and 18.96 s on average, respectively, and together account for the mean context reconstruction time of 21.72 s. Docker/GDB collection is the dominant substep. Mean LLM analysis latency is 6.63 s. Context measurements were collected serially, with up to three replay/debugger attempts and a 15 s per-attempt timeout. Failed and timed-out attempts are included.

Table 13 Mean runtime of the analysis stages.

Stage	Mean time
Sanitizer parsing	0.67 ms
Rule-guided attribution	0.10 ms
Representative selection	0.01 ms
Crash record lookup	0.32 s
Seed recovery and binary resolution	0.18 ms
Static context extraction	1.18 ms
Direct replay	2.44 s
Docker/GDB context collection	18.96 s
Context reconstruction (overall)	21.72 s
Evidence packaging and prompt construction	0.34 s
LLM analysis	6.63 s
End-to-end representative analysis	28.69 s

The mean end-to-end latency across the 96 representatives is 28.69 s, covering context reconstruction, evidence packaging and prompt construction, and LLM analysis. Context and prompt times come from the timing measurements, while LLM latency comes from the archived responses for the same representatives. Because the context measurements were collected serially, the 96 representatives require approximately 45.9 minutes of processing in total, which is the sum of the per-representative mean over the evaluated representatives.

The evaluated target API sets range from 12 to 60. cJSON (60 APIs) and libpcap (30 APIs) both reduce their respective sets of 78 and 115 reports to four LLM submissions, with mean LLM latencies of 6.17 s and 6.23 s per representative. In comparison, faad2 (12 APIs) reduces 151 reports to nine submissions, with a mean LLM latency of 7.72 s. Across all 14 libraries, project-level mean LLM latency ranges from 5.52 s to 7.72 s. These results support the potential scalability of representative-based, crash-local analysis to untested libraries with larger API sets. The analysis workload for such libraries would depend on representative count and per-representative replay/debugger cost.

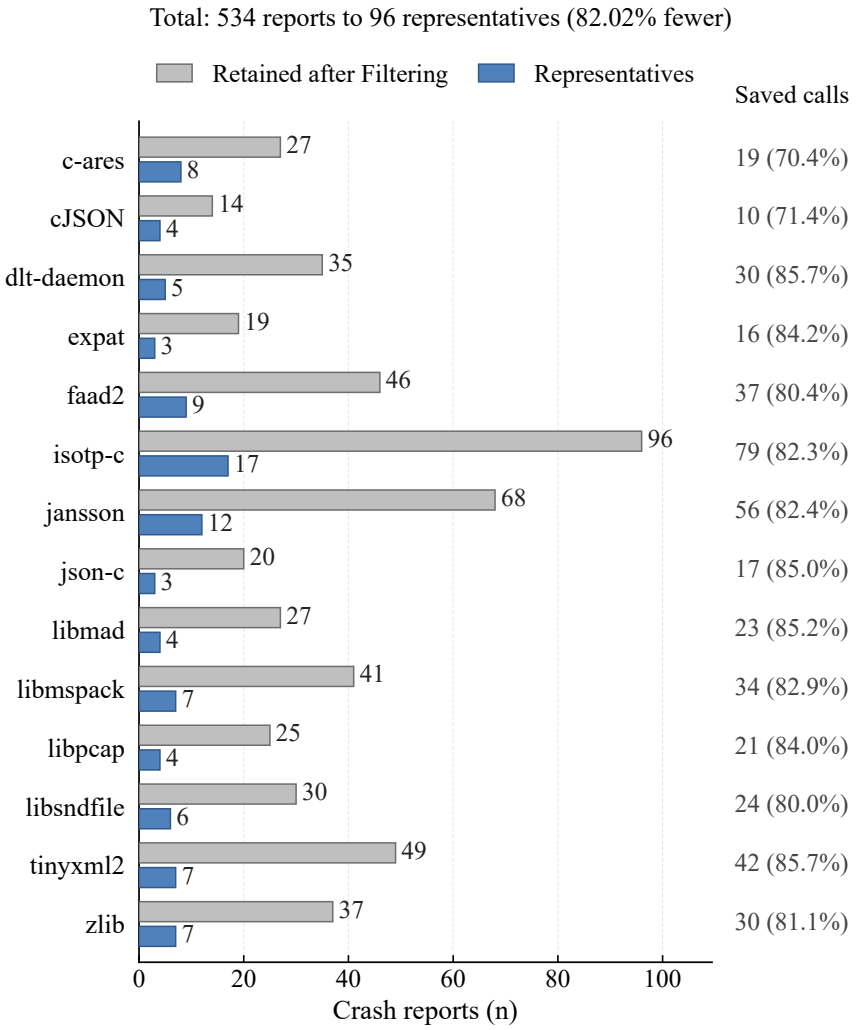


Fig. 4 Reduction in crash reports achieved by representative selection.

Figure 4 shows that V1 submits all 534 reports retained after rule-guided crash attribution to the LLM, whereas V2 submits only 96 representatives and removes 438 repeated or redundant reports.

Representative selection therefore reduces LLM submissions by 82.02% relative to V1. Only the prediction of each representative is propagated through the saved mapping, while every report retains its own ground-truth label and TP, FP, TN, and FN are computed against that label. Consequently, V2 preserves the overall attribution metrics of V1 while requiring substantially fewer LLM submissions. Although each V2 request contains richer analysis content than an individual raw report, reducing the number of LLM submissions from 534 in V1 to 96 in V2 substantially lowers the overall LLM invocation cost as measured by the number of LLM submissions.

To evaluate group-level error propagation, we flip each representative prediction individually and recompute the report-level metrics. The maximum absolute F1 change is 0.0174; the worst-case flip reduces F1 from 0.8379 to 0.8205 and affects less than 1% of report-level decisions.

To compare representative selection directly, we apply ReBucket to the same 534 reports retained after rule-guided crash attribution. CrashSage selects 96 representatives, whereas ReBucket selects 103 cluster representatives, corresponding to reductions of 82.02% and 80.71%, respectively. Thus, ReBucket requires seven additional LLM submissions relative to CrashSage on this workload. At the pairwise level, ReBucket preserves 1,450 of the 1,476 report pairs that CrashSage groups together and splits the remaining 26, whereas no pair is grouped by ReBucket alone. The pairwise Jaccard agreement is 0.9824. Igor and CrashLocator are not used as external references because both operate on crash-corpus formats and debug-information settings that differ from the ASan+UBSan reports used here; applying them would change the corpus and the attribution target. ReBucket operates on stack traces and applies to the retained reports without changing the corpus, so it serves as the reproducible external reference for the selection stage.

Figure 5 shows ReBucket's retained reports and representatives for each library in the same format as the CrashSage results. The representative counts differ for cJSON, with 4 for CrashSage and 9 for ReBucket, and libpcap, with 4 and 6, respectively. The other 12 libraries have the same representative counts under both methods. Each reduction is computed relative to that library's retained reports.

We audit whether representative selection merges distinct target reports in two complementary views. The structural audit covers all 215 target reports and groups them by project, sanitizer bug type, first target-project frame, and crash function. These reports form 42 representative groups over 16 structural fingerprints, and no group mixes two fingerprints, so all 16 structural classes remain represented after selection. The CVE audit covers the 80 reports with explicit public CVE identifiers. These reports form 19 groups over 7 CVEs, and no group mixes two CVE identifiers, so all 7 CVEs remain represented. Neither audit merges distinct target reports. The structural fingerprint is an auditable crash-site proxy rather than an independent manual root-cause label.

The 80 CVE-linked reports are a subset of the 215 target reports; the remaining 135 reports comprise 66 author-confirmed public driver+PoC cases without a CVE identifier, 43 targets from dlt-daemon and isotp-c, and 26 targets from the completed audit.

The reduction varies across libraries because the generated crashing inputs contain different levels of repetition. In response to RQ2, representative selection reduces repeated LLM analysis over the retained reports by keeping one report for each group of crashes with matching exact or stack signatures.

5.5 Evaluation of Crash-Context Reconstruction

To evaluate crash-context reconstruction, we compare V2 and V3 on the same 1,217 crash reports. Both variants use rule-guided crash attribution and representative selection, while only V3 includes

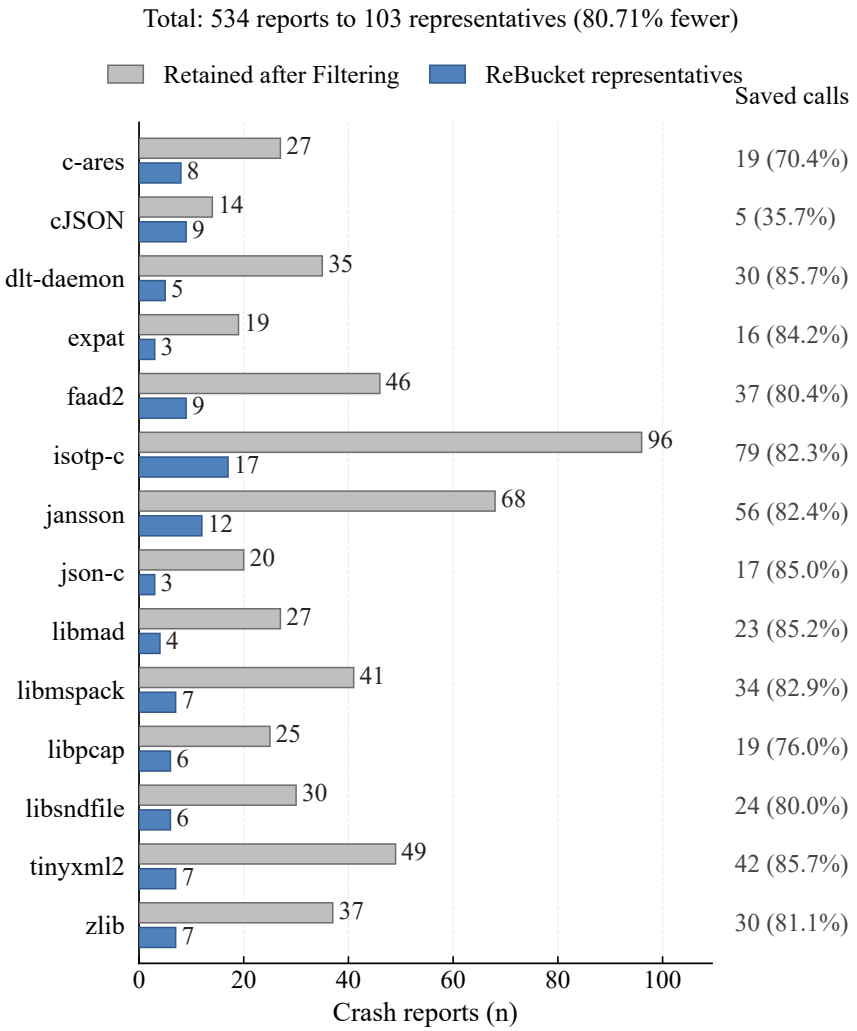


Fig. 5 Reduction in crash reports achieved by ReBucket.

crash-context reconstruction. With the ground-truth labels held fixed, this comparison evaluates the effect of adding crash-context reconstruction to V2.

Table 14 Effect of crash-context reconstruction.

Variant	LLM Calls	TP	FP	TN	FN	Precision	Recall	F1
V2	96	155	108	894	60	0.5894	0.7209	0.6485
V3	96	186	75	927	29	0.7126	0.8651	0.7815

The experimental results are shown in Table 14 and Table 15. Compared with V2, V3 achieves precision 0.7126, recall 0.8651, and F1 0.7815 under the ground-truth labels. This improvement is associated with the localization evidence added by crash-context reconstruction. A crash may become visible inside a target-component API even when the faulty state originates from the generated driver. Static code context links the visible crash site to the driver-to-component call path, while runtime context records the state observed during execution. In response to RQ3, these static and runtime evidence sources reduce the 108 false positives observed in V2 to 75, remove 31 of its 60 false negatives, and help distinguish the visible crash location from the origin of the faulty state under the ground-truth labels.

Table 15 Per-library effect of crash-context reconstruction.

Library	V2			V3		
	Prec.	Recall	F1	Prec.	Recall	F1
c-ares	0.5000	1.0000	0.6667	0.5000	1.0000	0.6667
cJSON	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
dlt-daemon	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
expat	0.6667	0.1538	0.2500	0.0000	0.0000	0.0000
faad2	0.8947	0.5667	0.6939	1.0000	1.0000	1.0000
isotp-c	0.1951	1.0000	0.3265	0.5333	1.0000	0.6957
jansson	0.4000	0.9231	0.5581	0.3824	1.0000	0.5532
json-c	0.0000	0.0000	0.0000	0.6500	1.0000	0.7879
libmad	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
libmspack	1.0000	0.8333	0.9091	1.0000	1.0000	1.0000
libpcap	0.8421	0.8421	0.8421	0.6667	0.6316	0.6486
libsndfile	0.5200	1.0000	0.6842	0.8667	1.0000	0.9286
tinyxml2	0.5000	1.0000	0.6667	0.3571	0.7692	0.4878
zlib	0.2143	0.2308	0.2222	0.6842	1.0000	0.8125

At the project level, json-c increases from F1 0.0000 in V2 to 0.7879 in V3, while cJSON remains at 0.0000. This result indicates that crash-context reconstruction alone does not resolve all project-specific attribution failures. Nevertheless, the subsequent V4 results show that the final prompt design can further affect how the LLM uses the same evidence package. We examine this interaction in the next subsection.

5.6 Evaluation of Final-Prompt Sensitivity and LLM

To evaluate the effect of prompt design, we compare four nested prompt variants of the differential prompt on the same representatives under the same evidence package. P0 is the Clean direct prompt used in V3. P1 adds the two-hypothesis comparison, P2 adds the precondition check and `evidence_rationale`, and P3 adds the boundary rules used in V4.

Table 16 shows that F1 rises at every step, but the three steps do not act on the same error type. The direct prompt yields more conservative positive decisions because it requests a final attribution without requiring the model to compare target-component-defect and driver-misuse hypotheses. P1 is the only step that recovers missed target-component crashes, and recall stops rising after it. P2 and P3 recover no further true positives and remove false positives only, so precision keeps rising while recall stays fixed. The hypothesis comparison is therefore the component that governs recall,

Table 16 Nested prompt ablation on 1,217 crash reports.

Prompt	Precision	Recall	F1
P0 (V3)	0.7126	0.8651	0.7815
P1	0.7263	0.9256	0.8139
P2	0.7453	0.9256	0.8257
P3 (V4)	0.7654	0.9256	0.8379

and the two later components refine precision. The benefit of the differential prompt comes from this separation rather than from a single template change. The mean F1 of the four configurations over the six repeated runs is 0.7788, 0.8095, 0.8260, and 0.8295, respectively, which preserves the same ordering as the reported runs.

The project-level results in Table 17 provide a concrete example of this prompt sensitivity. Only cJSON, expat, and jansson change between V3 and V4; the other eleven projects are unchanged. cJSON and expat gain target attributions, while jansson improves precision with recall already at 1.0000.

Table 17 Per-library effect of final prompt design under Qwen3-Coder-Flash.

Library	V3			V4		
	Prec.	Recall	F1	Prec.	Recall	F1
c-ares	0.5000	1.0000	0.6667	0.5000	1.0000	0.6667
cJSON	0.0000	0.0000	0.0000	1.0000	1.0000	1.0000
dlt-daemon	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
expat	0.0000	0.0000	0.0000	1.0000	0.5385	0.7000
faad2	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
isotp-c	0.5333	1.0000	0.6957	0.5333	1.0000	0.6957
jansson	0.3824	1.0000	0.5532	0.6500	1.0000	0.7879
json-c	0.6500	1.0000	0.7879	0.6500	1.0000	0.7879
libmad	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
libmspack	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
libpcap	0.6667	0.6316	0.6486	0.6667	0.6316	0.6486
libsndfile	0.8667	1.0000	0.9286	0.8667	1.0000	0.9286
tinyxml2	0.3571	0.7692	0.4878	0.3571	0.7692	0.4878
zlib	0.6842	1.0000	0.8125	0.6842	1.0000	0.8125

The cJSON result illustrates how representative-level decisions translate into report-level metrics. Its 78 reports contain six target and 72 non-target reports. Filtering retains 14 reports, which map to four representatives; all six target reports share one representative. With the evidence package and mapping fixed, this representative changes from non-target under V3 to target under V4, and its prediction is propagated to the six group members. The remaining 72 reports keep their non-target decisions. Consequently, TP changes from 0 to 6 and FN from 6 to 0, while FP remains 0 and TN remains 72, yielding the F1 change from 0.0000 to 1.0000. In the P1 configuration reported in Table 16, the same representative already changes to target after the two-hypothesis comparison is

added. Thus, the large project-level change reflects a prompt-sensitive representative decision shared by all six positives, rather than six independent model decisions.

Run-to-run stability. The four prompt configurations defined in the nested prompt ablation were frozen before evaluation and shared across projects. No project names, expected CVE identifiers, or ground-truth labels were inserted into the templates. We repeated each configuration six times at temperature 0 and report decision variability at both the run and report levels in Table 18. For P1 and P2, the reported values are the median-F1 runs of the six repetitions, whereas P0 and P3 use the formal V3 and V4 endpoints. The P2 and P3 intervals overlap, so their difference is interpreted as an observed effect rather than a statistically significant improvement.

Table 18 Six-run stability of the nested prompt ablation.

Prompt	Mean F1	SD	95% run-bootstrap interval	Reports changed across runs
P0	0.7788	0.0051	[0.7741, 0.7815]	9 / 1,217
P1	0.8095	0.0063	[0.8046, 0.8139]	13 / 1,217
P2	0.8260	0.0100	[0.8178, 0.8342]	14 / 1,217
P3	0.8295	0.0085	[0.8220, 0.8355]	23 / 1,217

The repeated results therefore show limited report-level variation even at temperature 0: at most 23 of the 1,217 decisions change across the six runs of any configuration.

CrashSage-V4 outperforms V3 on the same 1,217 crash reports; the quantitative comparison is reported in Table 16.

Furthermore, to evaluate whether the effectiveness of CrashSage-V4 depends on the final LLM backend, we repeat the V4 experiment with five additional LLMs. All models analyze the same dataset with the same V4 evidence package and differential prompt: the representative crashes, reconstructed contexts, prompt configuration, representative mapping, and ground-truth labels are held fixed, and only the final LLM is replaced. Each model is evaluated at temperature 0 on the same 96 representatives, and the decisions are expanded to the same 1,217 crash reports.

The primary CrashSage-V4 model is `qwen3-coder-flash`, accessed on 1 November 2026 through the OpenAI-compatible endpoint https://[workspace-id].cn-beijing.maas.aliyuncs.com/compatible-mode/v1. This model was selected on four engineering metrics: general and code-analysis capability, long-context capacity, service stability and response latency, and API cost.

The alternative backends are `qwen-flash`, `qwen3.8-flash`, `deepseek-flash`, `gpt-5.6-sol`, and `glm-5.3-flash`, accessed between 30 October and 1 November 2026. Each backend was accessed through the API provider under its exact model identifier. Model identifiers and access dates are reported so that each evaluated configuration can be identified. All configurations use `temperature=0`.

Table 19 shows that, with the V4 evidence package and differential prompt held constant, the evaluated backends achieve closely matched overall performance, with F1 ranging from 0.8240 to 0.8379 and recall from 0.9163 to 0.9256. The three backends from the DeepSeek, GPT, and GLM families reach 0.8354, 0.8354, and 0.8312, which places them inside this range, so the observed V4 behavior is not specific to the Qwen series. Backend selection can therefore follow deployment requirements while preserving the overall method behavior.

Table 19 Validation results of different LLM backends under the same V4 configuration.

LLM Backend	Precision	Recall	F1
Qwen3-Coder-Flash (default V4)	0.7654	0.9256	0.8379
Qwen-Flash	0.7425	0.9256	0.8240
Qwen3.8-Flash	0.7636	0.9163	0.8330
DeepSeek-Flash	0.7645	0.9209	0.8354
GPT-5.6-Sol	0.7645	0.9209	0.8354
GLM-5.3-Flash	0.7606	0.9163	0.8312

5.7 Case Studies

To complement the quantitative results above, we present three case studies that illustrate how CrashSage prepares structured crash evidence before final LLM analysis. They are selected to match the three evaluation concerns studied above: false-alert judgment, efficiency, and crash localization.

Table 20 summarizes three case studies corresponding to the three evaluation concerns. In Case 1, the sanitizer reports a `heap-buffer-overflow`, but the top crashing frame and heap-boundary evidence are both confined to the generated driver. The driver allocates a 20-byte `second_str` buffer and then writes the terminator after copying fuzz input into that buffer, so the faulty state is created before any target-component behavior needs to be considered. CrashSage therefore treats this report as `likely_driver_bug` and filters it before LLM analysis, illustrating how rule-guided attribution prevents a driver memory error from being counted as a target-component crash candidate.

In Case 2, the libpcap reports form a repeated SEGV group rather than independent failures. The raw reports share similar crash-site evidence and stack signatures, and CrashSage records repeated reports in the `driver_30` group as duplicates of selected representative reports. Representative selection therefore skips duplicate LLM submissions and reuses the representative mapping, providing a concrete instance of the RQ2 result: redundant SEGV reports are collapsed by signature evidence before final analysis.

Table 20 Case studies for CrashSage evaluation

Evidence	Case 1: False-alert judgment	Case 2: Efficiency	Case 3: Crash localization
Crash report	cJSON `driver_4_crash_1`	libpcap `driver_30` group	json-c `driver_13_crash_1`
Failure identity	`heap-buffer-overflow`	repeated `SEGV`	`SEGV` wild write
Attribution evidence	top crashing frame: driver	shared crash-site / stack signature	project frame in `printbuf_memappend`
Context or mapping	20-byte driver heap region	duplicate–representative pair; 10+ reports	state-sensitive `printbuf` context; replay confirmed
Pipeline output	`likely_driver_bug`; filtered	duplicate skipped; representative reused	context-supported `is_api_bug=true` result

In Case 3, the raw prompt-based analysis does not return `is_api_bug=true` for the json-c crash, although the crash is labeled true under the ground-truth labels. CrashSage changes this decision after

reconstructing the state-sensitive context around `printbuf_memappend`. The context records that the driver-created `printbuf` reaches the library with boundary-sensitive position and size fields. Replay confirms that this state consistently reaches the same failing library path. The reconstructed data flow shows that the generated driver directly sets `pb->bpos` near `INT_MAX` while keeping `pb->size` small before invoking `printbuf_memappend`. The additional static and runtime context changes the raw-report-level negative result into an `is_api_bug=true` result consistent with the target ground-truth label.

6 Limitations

The evaluation covers third-party-library components of connected-vehicle software. Full vehicle stacks, ECU integration, and on-vehicle deployment are not exercised, so the results describe component-level crash adjudication rather than vehicle-level behavior.

The current implementation follows one LLM-assisted fuzz-driver generation workflow with ASan+UBSan instrumentation. This keeps the comparison controlled, but other driver generators, prompt strategies, or sanitizer configurations may change the proportion of driver-induced crashes, duplicate reports, and target-component candidates.

Rule-guided crash attribution relies on deterministic rules over sanitizer signals and stack-frame roles, so its coverage is bounded by the sanitizer information and stack structure available in each report. Crash-context reconstruction depends on replaying the crashing input in a stable execution environment; when replay fails or the signature does not match, CrashSage falls back to the normalized report and static context, which may reduce localization confidence.

7 Conclusion

CrashSage introduces an evidence-guided crash-adjudication pipeline for LLM-assisted fuzzing workloads. By applying rule-guided crash attribution, representative selection, and crash-context reconstruction before final LLM analysis, it replaces direct reasoning over raw crash reports with analysis over filtered, representative, and reconstructed crash evidence. In particular, representative selection reduces repeated crash instances before LLM analysis, and crash-context reconstruction combines static code context with replay-consistent runtime context for the selected representatives to distinguish the visible crash site from the origin and propagation of the faulty state. Evaluation on 1,217 crash reports collected from 14 third-party open-source libraries, with all 1,217 reports used in the main evaluation, shows that CrashSage achieves 76.54% precision, 92.56% recall, and an 83.79% F1 score under ground-truth labels. The full pipeline also reduces end-to-end LLM submissions from 1,217 to 96 (92.11%). Rule-guided crash attribution and representative selection each require less than 10 ms per report, whereas crash-context reconstruction requires approximately 21.72 s and final LLM analysis uses 96 API calls. These results demonstrate that CrashSage improves attribution quality while substantially reducing redundant LLM submissions. Future work will extend evaluation to additional driver-generation workflows and sanitizer configurations.

Acknowledgements. Not applicable.

Authors' contributions. Gaoqing Guan contributed to conceptualization, methodology, implementation, evaluation, and writing of the original draft. Ying Zhang, Letian Sha, and Huiran Yang contributed to experimental design, validation, result analysis and interpretation, and manuscript review and editing. All authors read and approved the final manuscript.

Funding. Not applicable.

Availability of data and materials. The data supporting the findings of this study are available from the corresponding author upon reasonable request.

Declarations

Competing interests. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Bernardini C, Asghar MR, Crispo B (2017) Security and privacy in vehicular communications: Challenges and opportunities. Vehicular Communications 10:13–28. https://doi.org/10.1016/j.vehcom.2017.10.002

Böhme M, Pham VT, Nguyen MD, et al (2019) Directed greybox fuzzing. Communications of the ACM 62(9):90–97. https://doi.org/10.1145/3339252

Checkoway S, et al (2011) Comprehensive experimental analyses of automotive attack surfaces. In: Proceedings of the USENIX Security Symposium, URL https://www.usenix.org/conference/usenix-security-11/comprehensive-experimental-analyses-automotive-attack-surfaces

Dang Y, Wu R, Zhang H, et al (2012) ReBucket: A method for clustering duplicate crash reports based on call stack similarity. In: Proceedings of the International Conference on Software Engineering, pp 1084–1093, https://doi.org/10.1109/ICSE.2012.6227101

Deng Y, et al (2023) Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models. In: Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis, pp 423–435, https://doi.org/10.1145/3597926.3598067

Deng Y, et al (2024) Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries. In: Proceedings of the IEEE/ACM International Conference on Software Engineering, pp 70:1–70:13, https://doi.org/10.1145/3597503.3623343

Ding Z, Le Goues C (2021) An empirical study of OSS-Fuzz bugs. In: Proceedings of the IEEE/ACM International Conference on Mining Software Repositories, pp 131–142, https://doi.org/10.1109/MSR52588.2021.00026

Ghosh R, von Stockhausen HM, Schmitt M, et al (2025) CVE-LLM: Ontology-assisted automatic vulnerability evaluation using large language models. In: Proceedings of the AAAI Conference on Artificial Intelligence, pp 28757–28765, https://doi.org/10.1609/aaai.v39i28.35139

Huang Y, et al (2024) CrashTranslator: Automatically reproducing mobile application crashes directly from stack trace. In: Proceedings of the IEEE/ACM International Conference on Software Engineering, https://doi.org/10.1145/3597503.3623298

Jiang Z, et al (2021) Igor: Crash deduplication through root-cause clustering. In: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security, pp 3318–3336, https://doi.org/10.1145/3460120.3485364

Joshy AK, Le W (2022) FuzzerAid: Grouping fuzzed crashes based on fault signatures. In: Proceedings of the IEEE/ACM International Conference on Automated Software Engineering, pp 44:1–44:12, https://doi.org/10.1145/3551349.3556959

Ladisa P, Plate H, Martinez M, et al (2023) SoK: Taxonomy of attacks on open-source software supply chains. In: Proceedings of the IEEE Symposium on Security and Privacy, pp 1509–1526, https://doi.org/10.1109/SP46215.2023.10179304

Lemieux C, Sen K (2018) FairFuzz: Targeting rare branches to rapidly increase greybox fuzz testing coverage. In: Proceedings of the IEEE/ACM International Conference on Automated Software Engineering, pp 475–485, https://doi.org/10.1145/3238147.3238176

Lyu Y, et al (2024) Prompt Fuzzing for fuzz driver generation. In: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security, pp 3793–3807, https://doi.org/10.1145/3658644.3670396

Meng R, et al (2024) Large language model guided Protocol Fuzzing. In: Proceedings of the Network and Distributed System Security Symposium, URL https://www.ndss-symposium.org/ndss-paper/large-language-model-guided-protocol-fuzzing/

Petit J, Shladover SE (2015) Potential cyberattacks on automated vehicles. IEEE Transactions on Intelligent Transportation Systems 16(2):546–556. https://doi.org/10.1109/TITS.2014.2342271

Rodrigues IM, et al (2022) FaST: A linear time stack trace alignment heuristic for crash report deduplication. In: Proceedings of the IEEE/ACM International Conference on Mining Software Repositories, pp 270–281, https://doi.org/10.1145/3524842.3527951

Sanvito D, Arriciati G, Siracusano G, et al (2025) AutoCVSS: Assessing the performance of LLMs for automated software vulnerability scoring. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing: Industry Track, pp 564–575, https://doi.org/10.18653/v1/2025.emnlp-industry.38

Serebryany K, et al (2012) AddressSanitizer: A fast address sanity checker. In: Proceedings of the USENIX Annual Technical Conference, URL https://www.usenix.org/system/files/conference/atc12/atc12-final39.pdf

Shetty M, et al (2022) DeepAnalyze: Learning to localize crashes at scale. In: Proceedings of the IEEE/ACM International Conference on Software Engineering, pp 549–560, https://doi.org/10.1145/3510003.3512759

Stepanov E, Serebryany K (2015) MemorySanitizer: Fast detector of uninitialized memory use in C++. In: Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization, https://doi.org/10.1109/CGO.2015.7054186

Sun Q, et al (2022) A survey on cyber-security of connected and autonomous vehicles (CAVs). IEEE Transactions on Intelligent Transportation Systems 23(7):6240–6259. https://doi.org/10.1109/TITS.2021.3085297

Wang J, Huang Y, Chen C, et al (2024) Software testing with large language models: Survey, landscape, and vision. IEEE Transactions on Software Engineering 50(4):911–936. https://doi.org/10.1109/TSE.2024.3368208

Wu R, Zhang H, Cheung SC, et al (2014) CrashLocator: Locating crashing faults based on crash stacks. In: Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis, pp 204–214, https://doi.org/10.1145/2610384.2610386

Xia CS, et al (2024) Fuzz4All: Universal fuzzing with large language models. In: Proceedings of the IEEE/ACM International Conference on Software Engineering, pp 1547–1559, https://doi.org/10.1145/3597503.3639121

Xu H, et al (2025) CKGFuzzer: LLM-based fuzz driver generation enhanced by code knowledge graph. In: Proceedings of the IEEE/ACM International Conference on Software Engineering: Companion Proceedings, pp 243–254, https://doi.org/10.1109/ICSE-COMPANION66252.2025.00079

Yan J, et al (2024) Better debugging: Combining static analysis and LLMs for explainable crashing fault localization. CoRR abs/2408.12070. https://doi.org/10.48550/arXiv.2408.12070, arXiv:2408.12070